RICE UNIVERSITY

# NCCL and Interconnect Topology

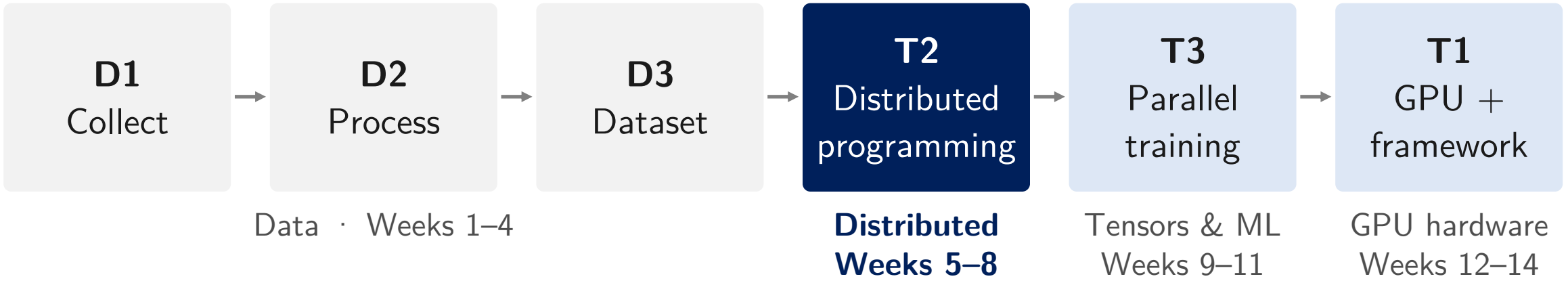
Topology, launching, hangs, and an $\alpha$–$\beta$ model of the network between servers

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 8, Lecture 2

# Where we are in the pipeline




**In our pipeline**

Today: collectives **across servers**, and the guided **two-server** benchmark with an $\alpha$–$\beta$ **fit** (Lab 4, due Week 9).

Last time: the ring all-reduce inside one server $\rightarrow$ today: NCCL and the links between servers

# Why go beyond one server?

**Train a 7.5-billion-parameter model on 2 trillion tokens.**
One server holds **8 GPUs**.

## How long does training take on one server?

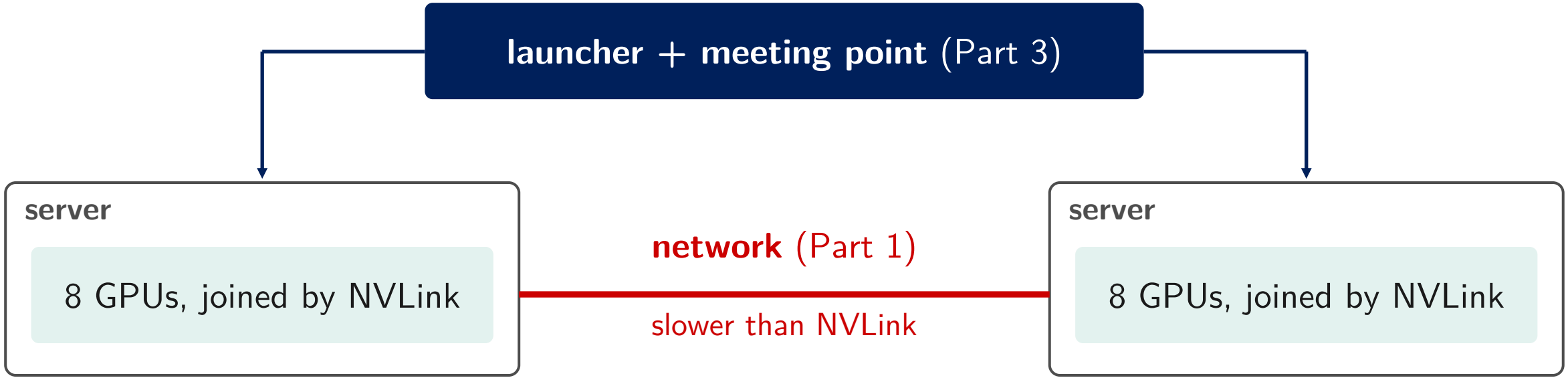



**Answer**

One server is **far too slow**. We must spread one job over **many servers** — and they must communicate fast.

Source: Estimate: 6 FLOP/parameter/token (Kaplan et al. 2020); 2T tokens (Llama 2); H100 bf16 peak (datasheet) at an assumed 40%.

# What a multi-server job needs




**+ an all-reduce that keeps most bytes on the fast NVLink (Part 2)**

**Definition**

A **server** (or **node**) is one machine. Going multi-server adds a **network** between them, a way to **start and connect** the processes, and **smarter collectives**.

# Where multi-server training shows up

- **Large models**: Llama 3 trained on up to **16K** H100 GPUs — over 2,000 servers.
- **PyTorch**: `torchrun` starts the processes on every server; NCCL carries the bytes.
- **Costs**: the network is $\approx$ **9×** slower than NVLink, and one failed machine stalls everyone.

**In our pipeline**

T2 and Lab 4: the guided two-server benchmark and an $\alpha$–$\beta$ fit. T3: the layout decides what crosses servers.

**Why go multi-server?**

Big jobs need **more GPUs than one server holds**. Going multi-server buys them, but adds a **slow link** and new ways to **hang**.

Source: Llama Team, "The Llama 3 Herd of Models," arXiv:2407.21783, 2024; H100 datasheet; 400 Gb/s NDR NIC.

# Learning goals for today

- **Describe** what connects GPUs **inside** a server and **between** servers, and how fast each link is.
- **Explain** why the **slowest link** sets the pace, and how a **hierarchical** all-reduce avoids it.
- **Explain** how a job starts on two servers, and the two common reasons it **hangs**.
- **Fit** an $\alpha$–$\beta$ model to two-server measurements and use it to predict.

# A puzzle to start

**Each GPU holds 1 GB of gradients; all-reduce them.**

A: 8 GPUs in one server. B: 16 GPUs in two servers.

**Is B faster, about the same, or slower than A?**

**Answer**

**Slower.** A $\approx$ 3.9 ms; B $\approx$ **37.5 ms** — about **10×** slower, with twice the GPUs.

Source: Model estimates. NVLink: H100 datasheet; network: 400 Gb/s NDR InfiniBand NIC (typical).

# Recap: the $\alpha$–$\beta$ model

- $\alpha$ (latency): the fixed start-up cost of one message.
- $\beta$ (bandwidth): the bytes per second a link moves.

$$\text{one message of } n \text{ bytes:} \quad T = \alpha + n \,/\, \beta$$

$$\text{ring all-reduce on } p \text{ GPUs, large } n: \quad T \approx 2(p-1)/p \cdot n \,/\, \beta$$

**Watch out**

Last lecture every link had the **same** $\beta$. Across servers they do not — which $\beta$ do we use?

# Today's plan

1

## Inside and between servers

What connects the GPUs, and how fast is each link?

2

## The slowest link sets the pace

If one link is 9× slower, how should the all-reduce change?

3

## Launching, and why jobs hang

How do processes on two machines find each other?

Build

## Two-server benchmark and $\alpha$–$\beta$ fit

Design it in pairs now; run it in Lab 4.

Part 1

# Inside and between servers

What connects the GPUs, and how fast is each link?

**1** · Inside and between servers

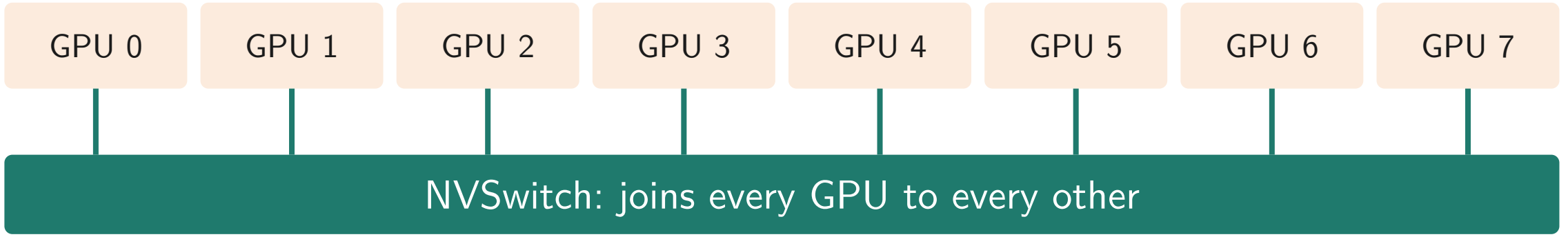
**2** · The slowest link sets the pace

**3** · Launching, and why jobs hang

**Build** · Two-server benchmark and $\alpha$–$\beta$ fit

# Inside one server

8 GPUs in one server



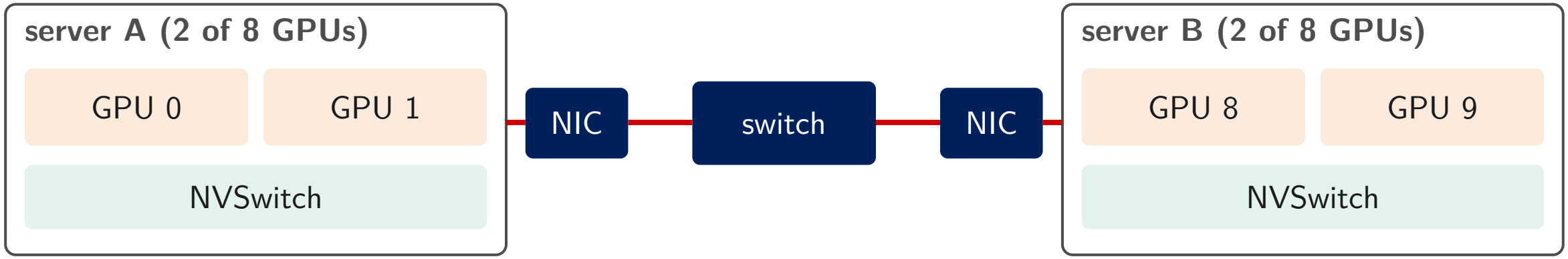

NVLink: **450 GB/s** each way, for every GPU

**Key idea**

**NVLink**: direct wires between GPUs. **NVSwitch**: a switch chip that joins them, so any two GPUs in the server talk at full speed.

Source: NVIDIA DGX H100 datasheet and user guide (8 × H100, 4 NVSwitch chips, 8 × ConnectX-7 at 400 Gb/s); H100 datasheet.

# Leaving the server: the network card




each NIC: **400 Gb/s** (a typical server has one per GPU)

$$400 \text{ Gb/s} \div 8 \text{ bits per byte} = 50 \text{ GB/s}$$

**Key idea**

**NIC** (network interface card): puts bytes on the cable. A **switch** forwards them to the right server. Network speeds are in **bits**: divide by 8.

Source: Typical NIC: 400 Gb/s NDR InfiniBand (InfiniBand Trade Association); NVLink: H100 datasheet (900 GB/s bidirectional).

# The gap: leaving the server is slow

per GPU, each way




**Watch out**

Leaving the server is **9× slower**. Every byte that crosses servers pays for it.

Source: Typical NIC: 400 Gb/s NDR InfiniBand (InfiniBand Trade Association); NVLink: H100 datasheet (900 GB/s bidirectional).

# Check yourself

Two minutes with a neighbour.

- A cluster has 200 Gb/s NICs. How many GB/s is that? How many times slower than NVLink?
- GPU 0 sends to GPU 5 in the same server, then to a GPU in another server. Which links?
- Why is a server with **one** NIC for 8 GPUs a problem?

Part 2

# The slowest link sets the pace

If one link is 9× slower, how should the all-reduce change?

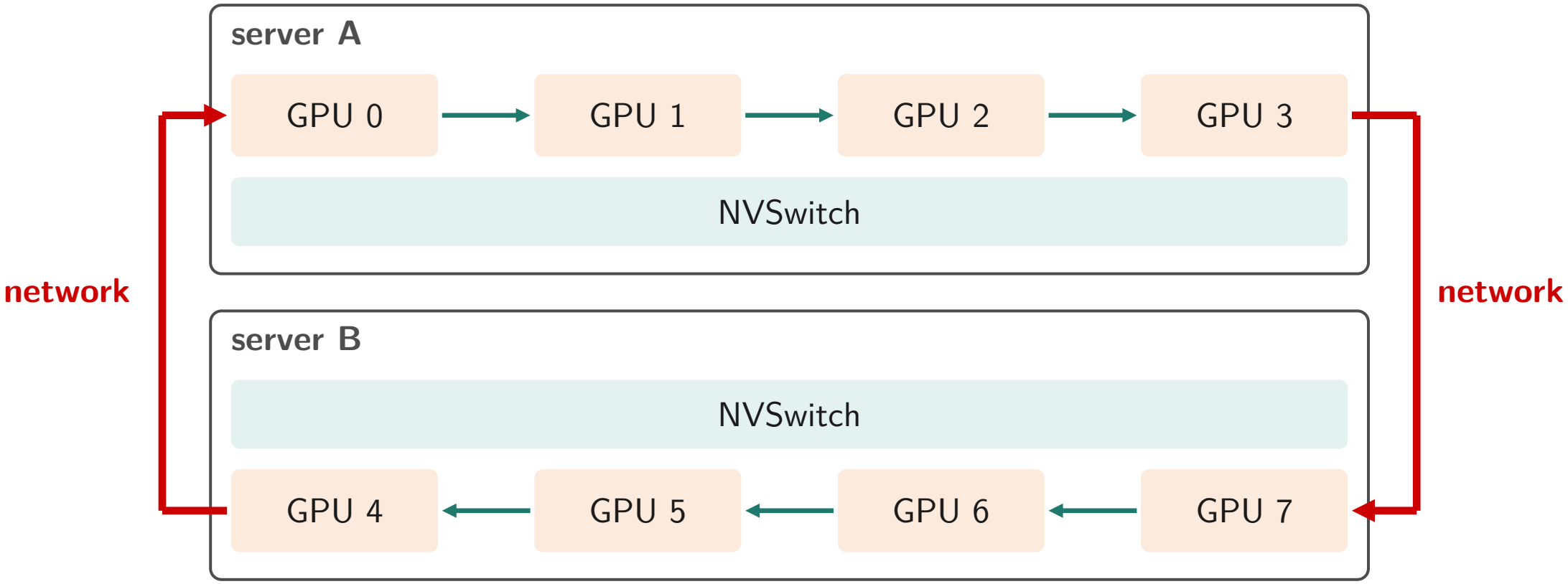

1 · Inside and between servers

**2** · The slowest link sets the pace

**3** · Launching, and why jobs hang

**Build** · Two-server benchmark and $\alpha$–$\beta$ fit

# A flat ring across two servers




**Key idea**

**Flat ring**: one ring through all GPUs. Six hops are fast NVLink, two are the slow network.

# The slowest link sets the pace

- In every step, **all** GPUs send one piece at the same time.
- The step ends when the **slowest** send ends; the others wait.

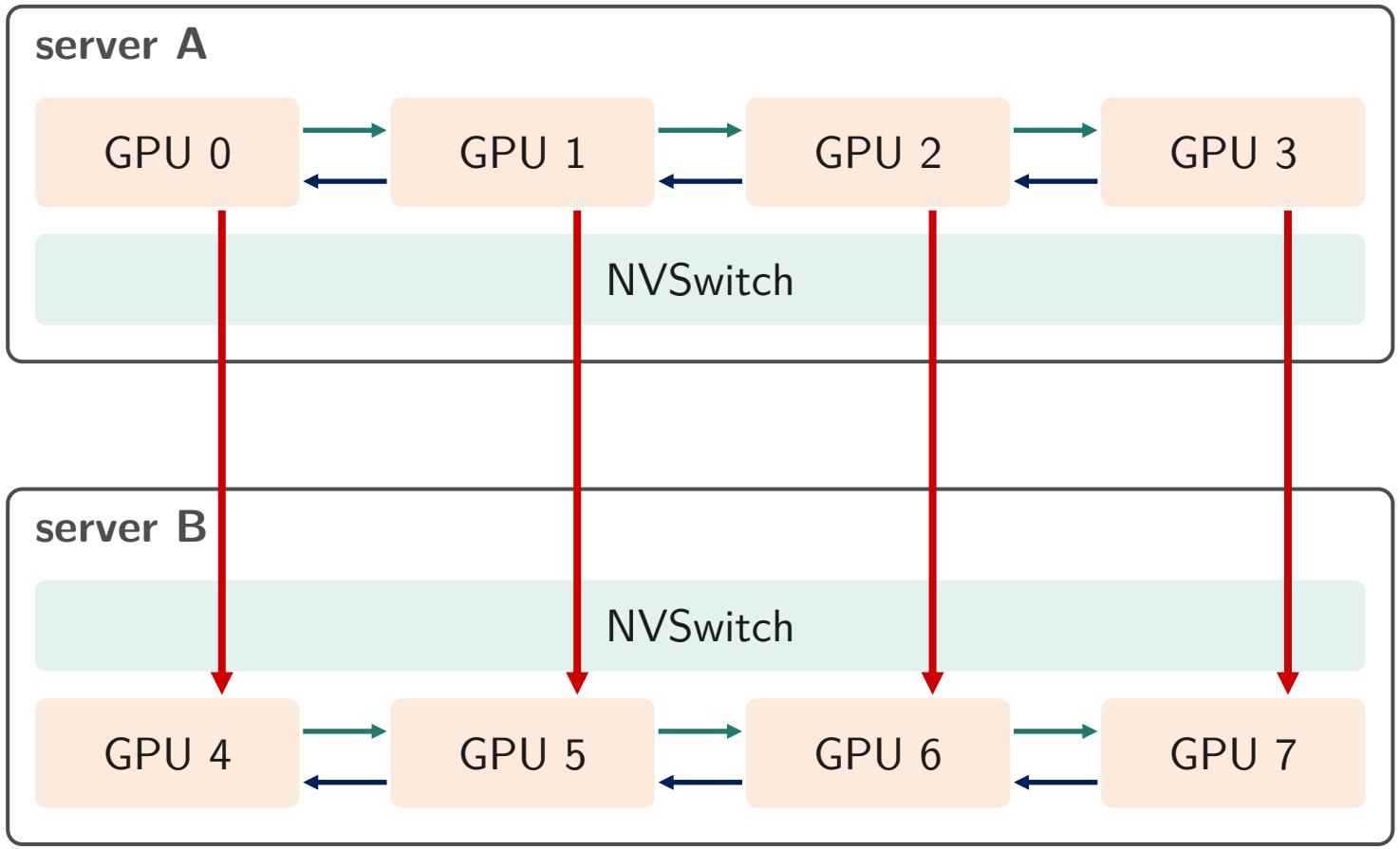
$$T \approx 2(p-1)/p \cdot n \;/\; \beta(\text{slowest})$$

**Result**

Puzzle B: $\beta(\text{slowest}) = 50$ GB/s, so T $\approx$ **37.5 ms**. The fast links sit idle most of each step.

Source: Typical NIC: 400 Gb/s NDR InfiniBand (InfiniBand Trade Association); NVLink: H100 datasheet (900 GB/s bidirectional).

# Hierarchical all-reduce: three phases




**1** · reduce-scatter inside each server (NVLink)

**2** · the k-th GPUs of A and B all-reduce slice k, each on its own NIC

**3** · all-gather inside each server (NVLink)

**Key idea**

Each NIC carries only $\mathbf{1/g}$ of the data (g = GPUs per server).

# One worked number: the puzzle again

puzzle B: 16 GPUs in two servers, 8 per server, 1 GB each




total $\approx$ **6.4 ms**

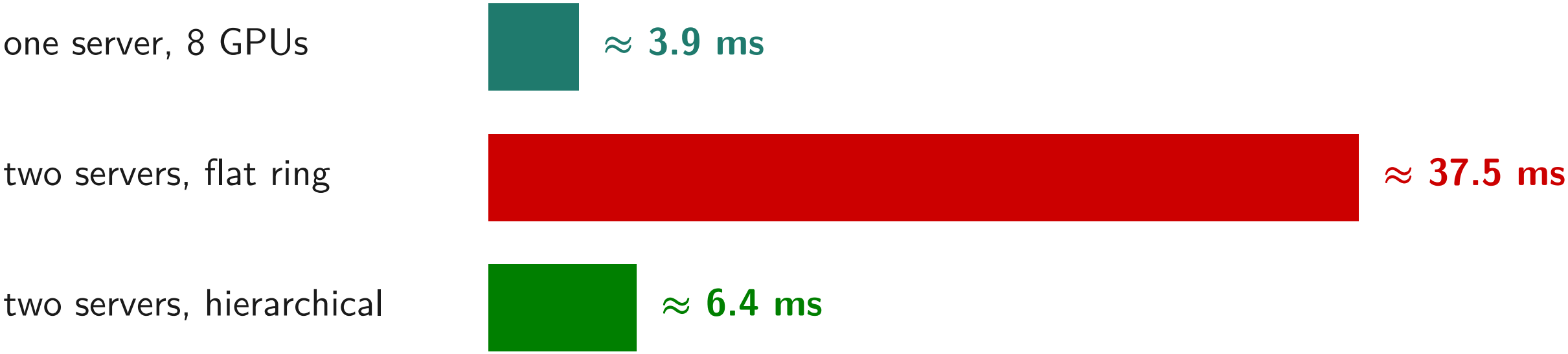
network phase: each NIC carries 1 GB $\div$ 8 $\rightarrow$ 2.5 ms

**Why it works**

Most of the work moves to NVLink. The slow network carries only **1/8** of the data per NIC.

Source: Typical NIC: 400 Gb/s NDR InfiniBand (InfiniBand Trade Association); NVLink: H100 datasheet (900 GB/s bidirectional). Model estimate.

# Back to the puzzle




**Answer**

A server-aware algorithm recovers most of the speed — **if** each GPU has its own NIC. NCCL does this for you.

Source: Typical NIC: 400 Gb/s NDR InfiniBand (InfiniBand Trade Association); NVLink: H100 datasheet (900 GB/s bidirectional). Model estimates.

# Check yourself

Two minutes with a neighbour.

- Redo the hierarchical estimate with **200 Gb/s** NICs (one per GPU).
- Why does a flat ring across two servers waste NVLink?
- Which traffic would you keep **inside** a server: the frequent or the rare?

Part 3

# Launching, and why jobs hang

How do processes on two machines find each other?

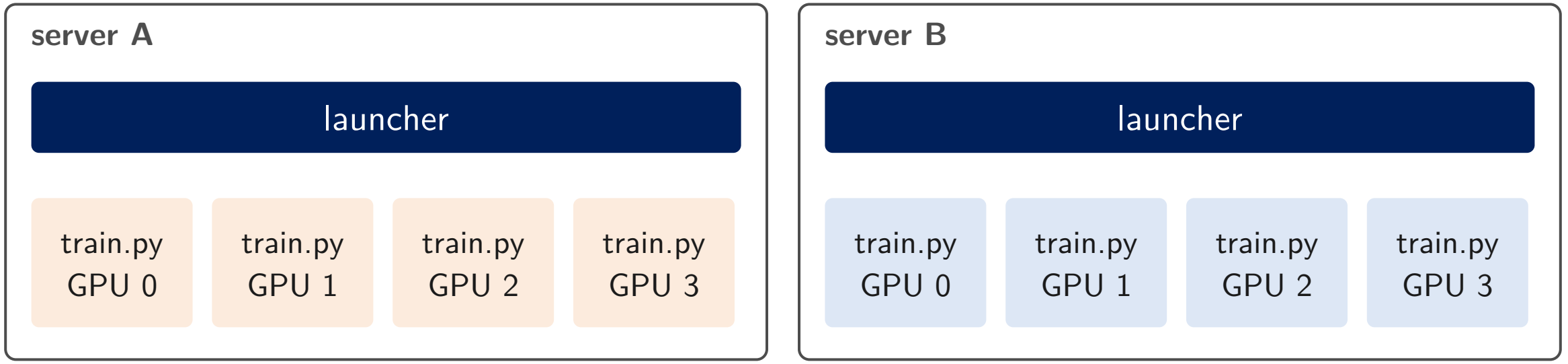
1 · Inside and between servers

2 · The slowest link sets the pace

**3** · Launching, and why jobs hang

**Build** · Two-server benchmark and $\alpha$–$\beta$ fit

# The same program on every server




one command, typed on each server (or run by the cluster's scheduler)

**Key idea**

**Launcher**: starts one copy of your script per GPU and tells each copy **who it is**.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

# Who am I? Node rank and local rank

**node rank**: which server · **local rank**: which GPU inside it

**server A** · **node rank 0**

| **rank 0** | **rank 1** | **rank 2** | **rank 3** |
|---|---|---|---|
| local 0 | local 1 | local 2 | local 3 |
| GPU 0 | GPU 1 | GPU 2 | GPU 3 |

**server B** · **node rank 1**

| **rank 4** | **rank 5** | **rank 6** | **rank 7** |
|---|---|---|---|
| local 0 | local 1 | local 2 | local 3 |
| GPU 0 | GPU 1 | GPU 2 | GPU 3 |

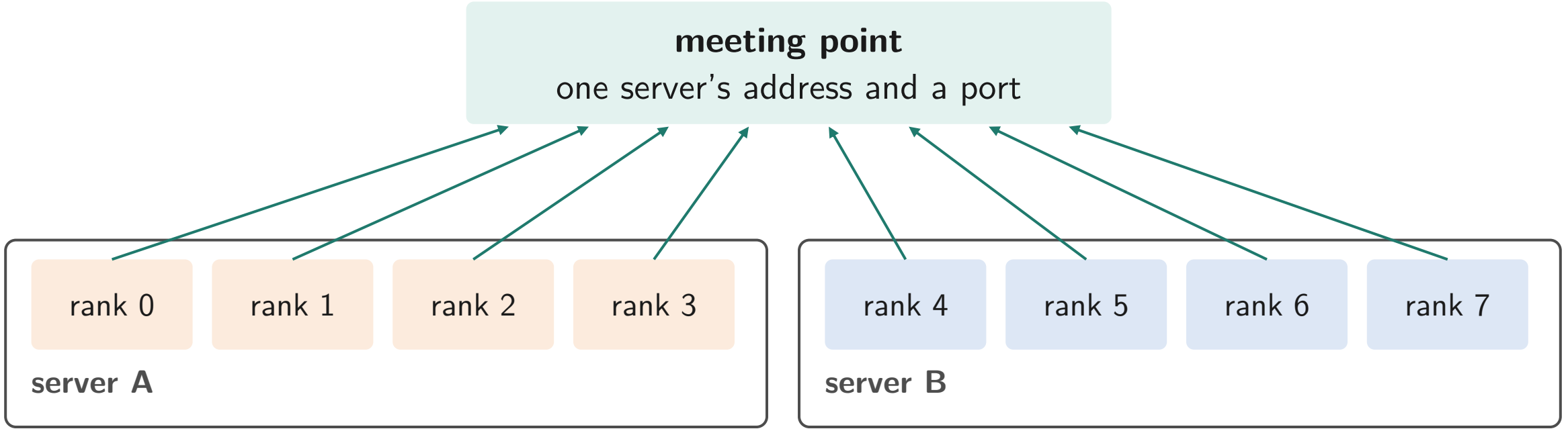
$$\text{rank} = \text{node rank} \times 4 + \text{local rank}$$

**Key idea**

Example: node 1, local rank 2 $\rightarrow$ rank $1 \times 4 + 2 = \mathbf{6}$, on GPU 2. The **world size** is the number of processes: 8.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

# Finding each other: a meeting point




each waits until **all 8** have arrived; then the job starts

**Watch out**

**Timeout**: how long a process waits before giving up with an error. If one server never starts, the others stop at the timeout.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

# The first lines of every script

```
local = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local)      # my GPU
dist.init_process_group(          # meet
    "nccl", timeout=timedelta(minutes=5))
rank = dist.get_rank()            # 0 .. 7
print(rank, local, socket.gethostname())
```

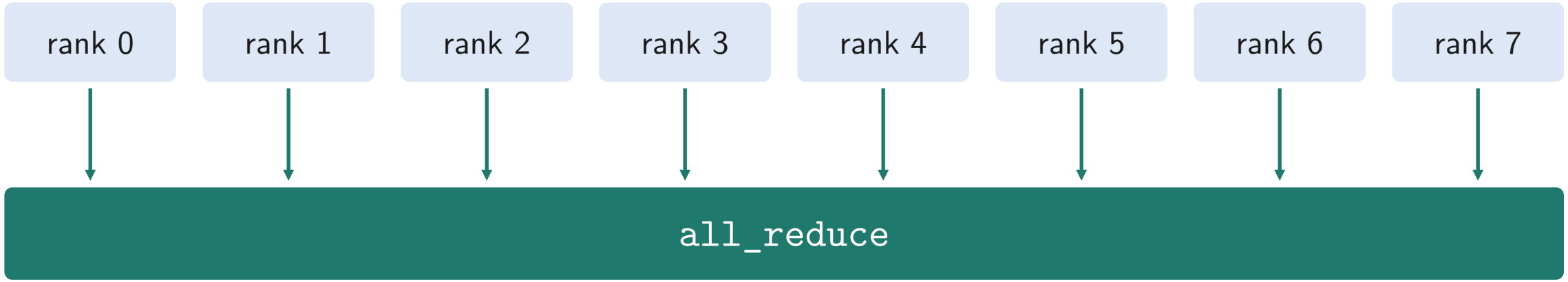
- Same file on one server or two.
- A **short** timeout: mistakes show up fast.
- Print who runs **where**.

**In our pipeline**

Lab 4, step 1: a table of rank, local rank, server, and GPU for **every** process.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

# A collective is a meeting

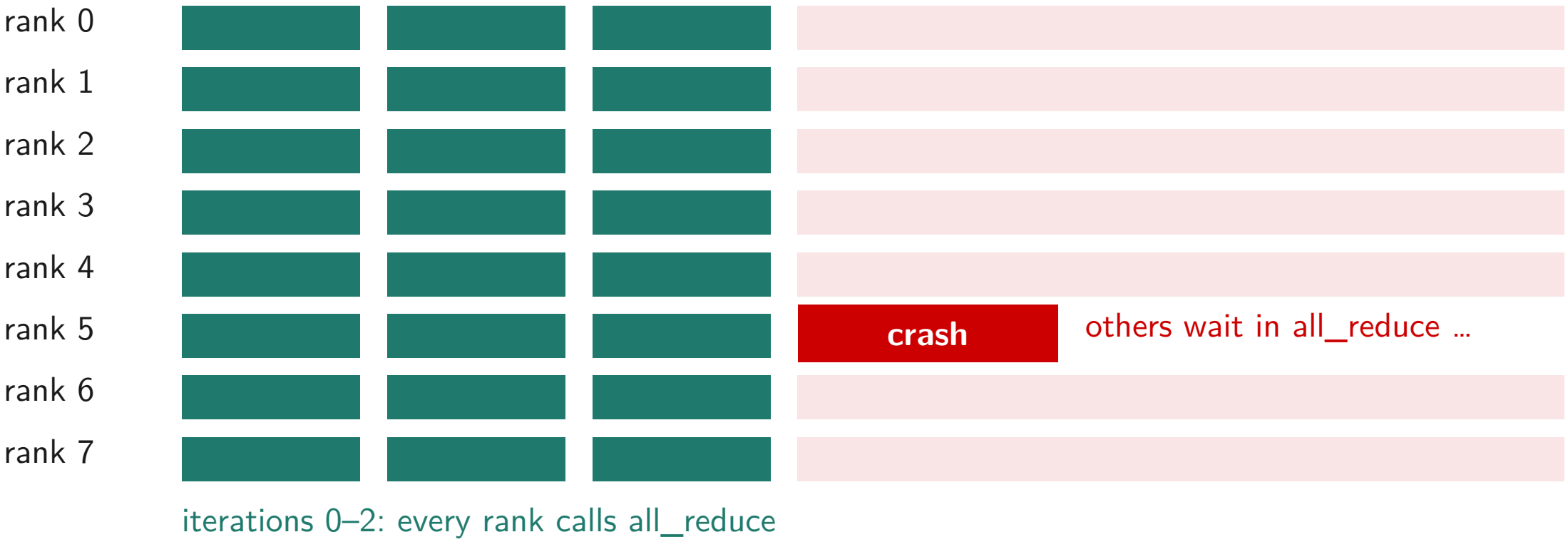



nobody leaves until **everyone** has arrived

**Watch out**

**Hang**: no error, no progress. Some rank never arrives — or arrives at a **different** collective.

# Hang 1: a crashed rank




**Watch out**

Read the log of the **first rank that failed**; the others only report the timeout.

# Hang 2: a different order of calls

| | ranks 0–6 | rank 7 (old copy of the code) |
|---|---|---|
| call 1 | `all_reduce(grads)` | `all_reduce(loss)` |
| call 2 | `all_reduce(grads)` | `all_reduce(grads)` |

call 1 meets call 1: gradients paired with a loss → hang or wrong sums

**The rule**

Every rank must call the **same collectives** in the **same order**, with the same sizes.

# A short debugging checklist

- **Start small**: 2 GPUs on one server, then 1 GPU on each of two servers.
- **Print** rank, local rank, server, and GPU from every process.
- **Same code and software** on every server.
- **Count collectives**: does every rank reach the same call?
- On a timeout, read the **first rank that failed**.

Source: NVIDIA NCCL User Guide: Environment Variables; Troubleshooting.

# Check yourself

Two minutes with a neighbour.

- 4 processes per server. Which rank and GPU does local rank 3 on node 1 get?
- Seven ranks print "step 41"; rank 2 has printed nothing since step 40. Where do you look?
- Only rank 0 runs `all_reduce(loss)` to print the loss. What happens?

IN-CLASS BUILD

# Two-server benchmark and $\alpha$–$\beta$ fit

Design it in pairs now; run it in Lab 4.


1 · Inside and between servers

2 · The slowest link sets the pace

3 · Launching, and why jobs hang

**Build** · Two-server benchmark and $\alpha$–$\beta$ fit

# Exercise A: design the two-server benchmark

**intra**
2 GPUs in one server
link: NVLink

**inter**
1 GPU on each of two servers
link: the network

**QUESTIONS**

(1) Why use exactly 2 GPUs in **both** runs?

(2) Which message sizes, from smallest to largest?

(3) How many repetitions, and which time do you report?

# Solutions A: the design

- (1) Same work in both runs: **only the link differs**.
- (2) **Tiny to very large**, spaced evenly on a log scale, so both $\alpha$ and $\beta$ show.
- (3) Warm up, time **many** runs, report the **slowest rank**; repeat the sweep.

**In our pipeline**

This is the guided two-server part of **Lab 4** and **T2**.

# Fitting $\alpha$ and $\beta$: a straight line

$$T(n) = \alpha + n \,/\, \beta$$

```
n = np.array(sizes, float)        # bytes
t = np.array(times)               # seconds
A = np.stack([np.ones_like(n), n], axis=1)
w = 1 / t                         # weights
(a, b), *_ = np.linalg.lstsq(A * w[:, None],
                             t * w, rcond=None)
alpha, beta = a, 1 / b
```

**RESULT**

Weight each point by $\mathbf{1/t}$. Without it the largest messages decide, and $\alpha$ comes out wrong.

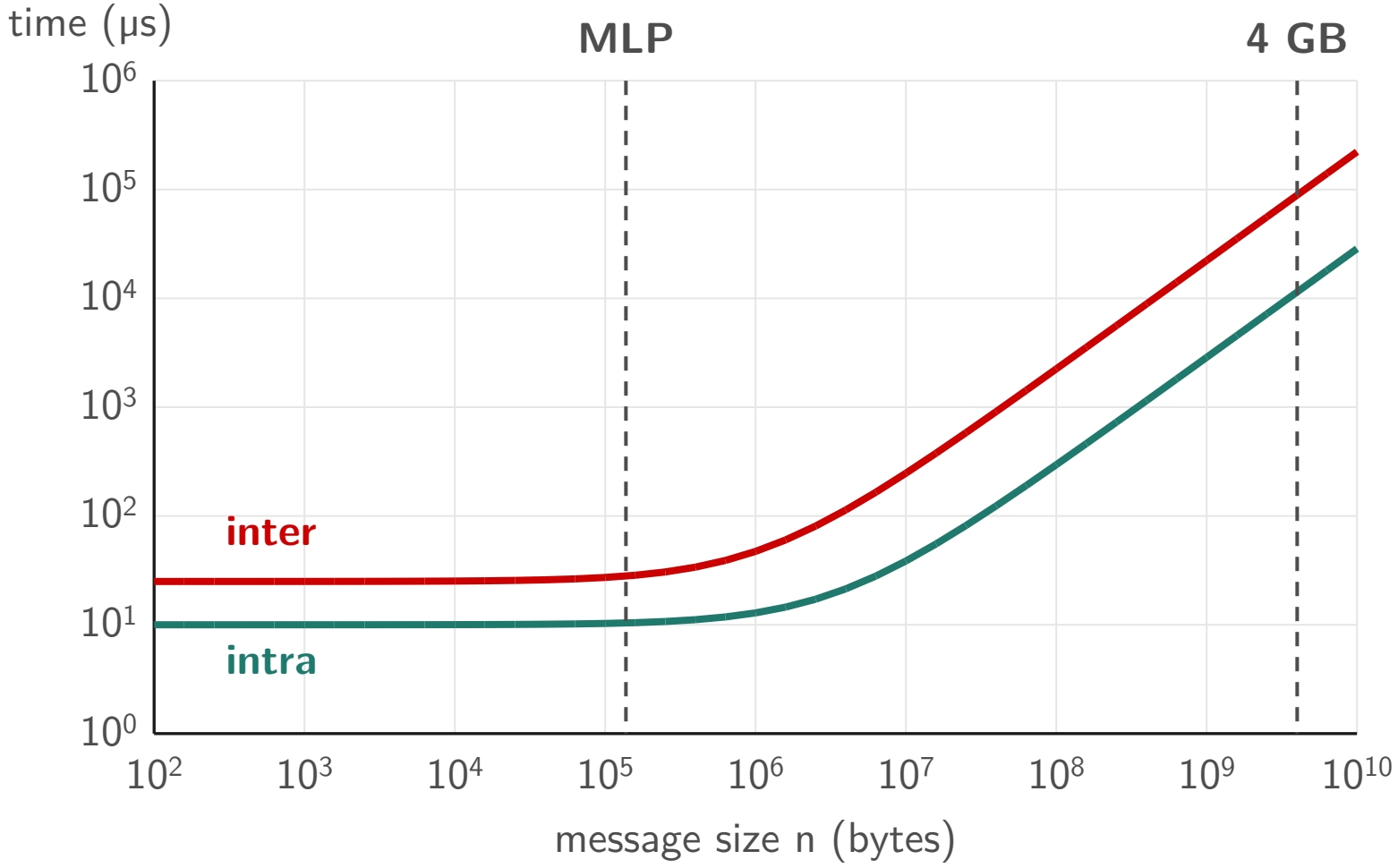
# Exercise B: predict with a fit

**GIVEN**

Inter-server fit: $\alpha = 25$ µs, $\beta = 45$ GB/s

- (1) Our MLP's gradient, 137,352 bytes: how long?
- (2) A 4 GB gradient: how long?
- (3) Which term, $\alpha$ or $n/\beta$, decides each answer?

# What the two fits look like




**READ IT**

**Flat**: time $\approx \alpha$.

**Sloped**: time $\approx n/\beta$.

The bend is at

$\mathbf{n^* = \alpha \cdot \beta}$.

Source: Model curves with assumed parameters, not measurements; replace with your Lab 4 fits.

# Solutions B: latency or bandwidth

- (1) MLP: $\approx$ **28 µs** — almost all $\alpha$.
- (2) 4 GB: $\approx$ **89 ms** — almost all $\mathbf{n}/\beta$.
- (3) Small messages pay for **latency**; large ones pay for **bandwidth**.

**Check**

Know your regime **before** optimizing: compare the message size n with $n^* = \alpha \cdot \beta$.

# Lab 4: what to hand in

- The **rank table**: rank, local rank, server, GPU for every process.
- Two sweeps, **intra** and **inter**, with the same sizes and harness.
- $\alpha$ **and** $\beta$ for each link, a log–log plot, and the two predictions.

**In our pipeline**

Report the **gap** between intra and inter, and explain it with $\alpha$, $\beta$, and the topology.

# Key takeaways

- Inside a server: **NVLink**, 450 GB/s per GPU. Between servers: a **NIC**, 50 GB/s — 9× slower.
- A ring runs at the speed of its **slowest link**.
- **Hierarchical** all-reduce: most work on NVLink; each NIC carries only 1/g of the data.
- Launch the **same script** on every server; rank = node rank × GPUs per server + local rank.
- Hangs: a **crashed rank**, or ranks calling collectives in a **different order**.
- Fit $\mathbf{T = \alpha + n/\beta}$ per link; $n^* = \alpha \cdot \beta$ separates latency- from bandwidth-bound.

# Next time, and readings

| When | What |
|---|---|
| This week | Midterm (Weeks 1–7) |
| Week 9 | T2 checkpoint; Lab 4 due |
| Week 9, Lecture 1 | Data parallelism: many GPUs, one averaged gradient |

**READ BEFORE WEEK 9**

PyTorch *Distributed Overview* and the DDP tutorial.

NCCL User Guide: *Troubleshooting*.